Instituto Superior Técnico

Elaborated in the context of the BLUE project

Automation of CAD design in Siemens NX

Emanuel Camacho - emanuel.camacho@tecnico.ulisboa.pt

May 2026

Contents

1 Introduction 2

2 NX setup 2

3 NX Journals 3

4 MATLAB pipeline 5

5 Results 6

6 References 7

1 Introduction

The CAD modelling of complex engineering parts can be a time consuming matter, this is especially true if some key design variables upon which almost all operations depend are subjected to a process of optimization and recalculation throughout the design phase.

This short document aims at providing a quick introduction into the process of CAD automation in the context of engineering design using, NX Journals, MATLAB and a specific CAD creation mindset.

2 NX setup

Before going about the automation of the CAD it is very important to first get familiar with the design process of the part in question :

- Which processes are you going to use (eg. revolve, extrude, etc.)?
- Which design variables do you want to change throughout the design process (eg. number of blades, inlet diameter etc.)?
- What is order of the operations that I want to make (eg. first create a pattern, then unite the bodies)?
- What finishing operations need to be performed (eg. edge blends etc.)?

All these questions may seem trivial at first but, if I want to design a simple sketch to revolve like this one below for the impeller base :

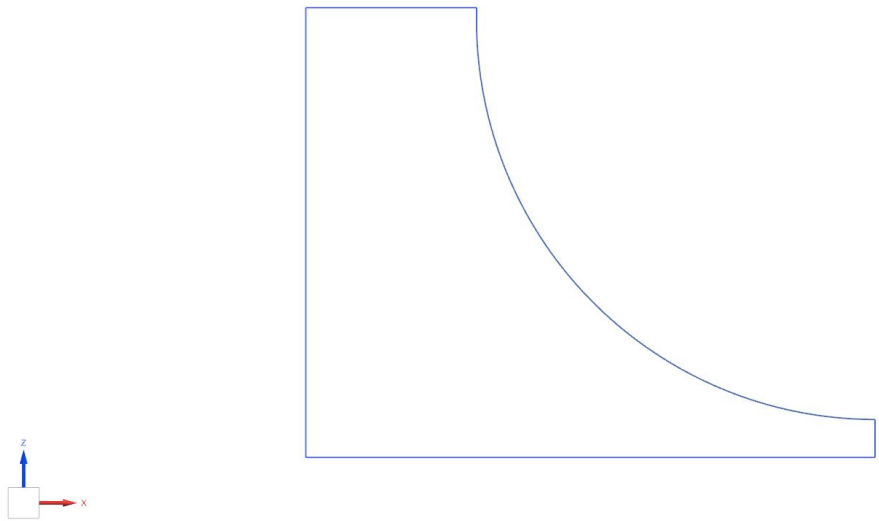

Figure 1: Revolve sketch for the impeller base

However, when actually designing it, we need to define the lines according to all variables that we want to change **but also the variables affected by those changes**. We do this via user expressions which can be opened by the shortcut *ctrl + E* in NX. In the context of this tutorial specifically, the variables were at first manually defined and then automatically changed via the NX Journals.

Here's a small taste of how a sketch looks after defining it fully, based on user defined expressions :

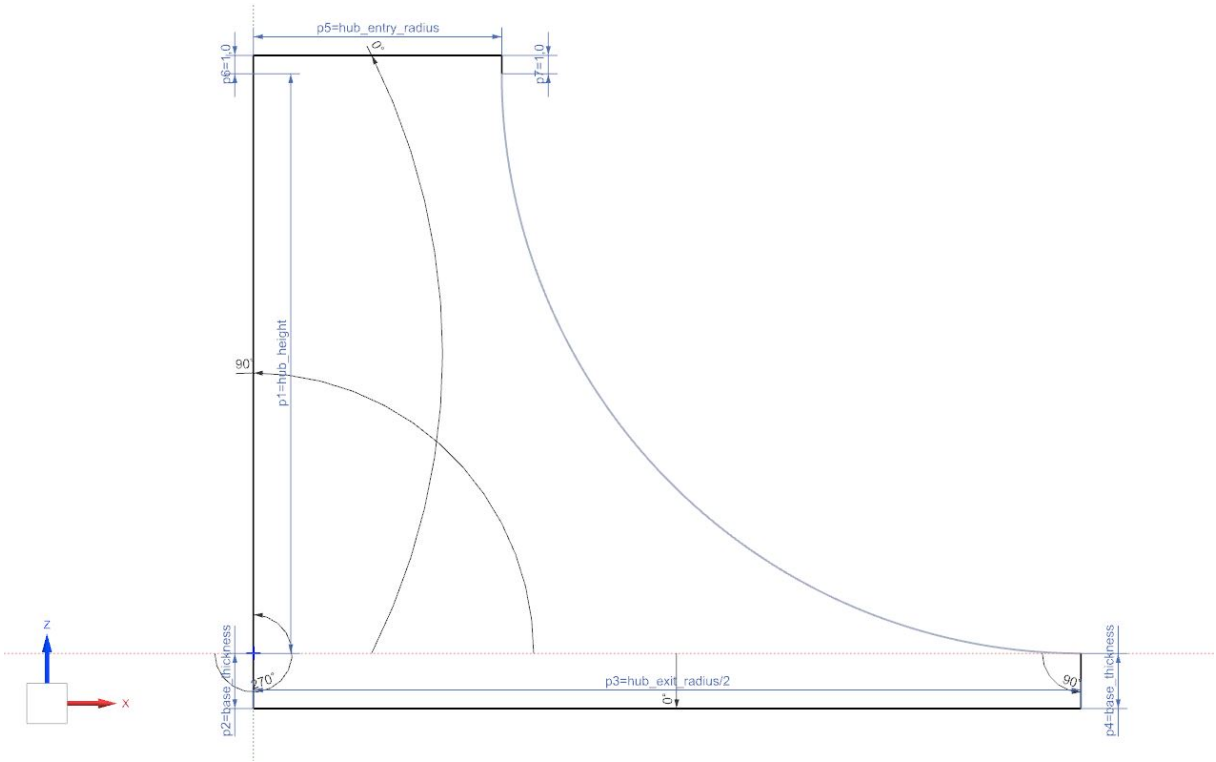

Figure 2: Expressions based sketch

3 NX Journals

NX Journals is a very powerful tool that allows us to run a script (in C, Python etc.) within the NX environment. This can be used in a variety of manners, importing points from a file (generated by MATLAB for example), drawing a curve with those points, redrawing the same curve if the points changed, change user defined expressions within NX etc.

For most IST aerospace engineering students, despite our familiarity with C from the bachelor's, Python will probably provide the safest bet when it comes to familiarity.

The first step is importing the NXOpen library as well as os (system files) and please remember that you **DO NOT** have to pip install the library, it will run within NX and not your default python environment :

```
import NXOpen
import NXOpen.Features
import os
```

The main function that will update your CAD will probably look something like this :

```
def main():
    the_session = NXOpen.Session.GetSession()
    base_path = r"C:\Users\Emanuel Camacho\Desktop\BLUE\matlabownworkk\FULL"
    part_path = r"C:\Users\Emanuel Camacho\Desktop\BLUE\NXfiles\impellerauto.prt"

    # Open Part
    base_part, load_status = the_session.Parts.OpenDisplay(part_path)
    work_part = the_session.Parts.Work

    # 1. SUPPRESS 3D FEATURES (Prevents external update crashes)
    to_suppress = [f for f in work_part.Features if any(x in f.Name.upper() for x in ["
    REVOLVE", "LOFT", "BLEND", "PATTERN"])]
    if to_suppress:
        work_part.Features.SuppressFeatures(to_suppress)

    # 2. UPDATE EXPRESSIONS
    param_file = os.path.join(base_path, "NX_parameters.txt")
    if os.path.exists(param_file):
        with open(param_file, 'r') as f:
            for line in f:
```

```
                if '=' in line:
                    name, val = line.split('=')
                    for expr in work_part.Expressions:
                        if expr.Name == name.strip():
                            expr.RightHandSide = val.strip()

    # 3. UPDATE CURVES
    update_spline_from_file(os.path.join(base_path, r"Hubcurvepoints\meridional_hub.txt"), "HUB_CURVE", work_part)
    sections = ["s000", "s017", "s033", "s050", "s067", "s083", "s100"]
    for i, s in enumerate(sections):
        p = os.path.join(base_path, "Bladecurvepoints", f"blade_section_{s}.txt")
        update_spline_from_file(p, f"BLADE_SECTION_{i}", work_part)

    # 4. SAVE (The 3D features remain suppressed so the save is guaranteed)
    mark_id = the_session.SetUndoMark(NXOpen.Session.MarkVisibility.Visible, "MATLAB_Update")
    the_session.UpdateManager.DoUpdate(mark_id)
    work_part.Save(NXOpen.BasePart.SaveComponents.TrueValue, NXOpen.BasePart.CloseAfterSave.FalseValue)
```

Its job is simple, it opens the session, looks for any files you wish to import and the part you wish to update, it opens the files and runs the functions you have defined. It finally saves the part while trying to avoid errors in building such as revolves using open sketches for example.

Note that in this case, another function is called, one that replaces the splines used for the blade curve redrawing while the other one updates the parameters in the user defined expressions :

```
def update_spline_from_file(file_path, feature_name, work_part):
    if not os.path.exists(file_path): return
    points_3d = []
    with open(file_path, 'r') as f:
        for line in f:
            try:
                c = [float(x) for x in line.replace(',', ' ').split()]
                if len(c) >= 3: points_3d.append(NXOpen.Point3d(c[0], c[1], c[2]))
            except: continue

    existing_feat = None
    for feat in work_part.Features:
        if feat.Name == feature_name:
            existing_feat = feat
            break

    builder = work_part.Features.CreateStudioSplineBuilderEx(existing_feat)
    builder.Type = NXOpen.Features.StudioSplineBuilderEx.Types.ThroughPoints
    builder.ConstraintManager.Clear()
    for pt in points_3d:
        nx_point = work_part.Points.CreatePoint(pt)
        gcd = builder.ConstraintManager.CreateGeometricConstraintData()
        gcd.Point = nx_point
        builder.ConstraintManager.Append(gcd)
    builder.Commit()
    builder.Destroy()
```

You'll find the full python file in the repository of the BLUE project.

If you wish to test out your journal that you've edited externally, for example in VS Code simply type edit journal in the NX search bar on the top right :

Figure 3: Edit journal location

There you can open your scripts and play them directly watching out for any errors that pop up according to your changes and functions.

4 MATLAB pipeline

The only thing left to do after the python code has already been written is to simply call it from MATLAB. This is done by sourcing the native external journal .exe as seen in line 18 and the python file you just wrote as seen in line 19. Finally, we run the whole thing by using the system function in line 23 and simply wait for the output. The code snippet can be found below :

```
1  % ========================================================================
2  %% FINAL EXPORT & TRIGGER NX
3  % ========================================================================
4
5
6
7  param_file = 'C:\Users\Emanuel Camacho\Desktop\BLUE\matlabownworkk\FULL\NX_parameters.txt
       ';
8  fid = fopen(param_file, 'w');
9  fprintf(fid, 'hub_entry_radius = %.6f\n', D1r*1000/2);
10 fprintf(fid, 'hub_height = %.6f\n', hub_height*1000);
11 fprintf(fid, 'hub_exit_radius = %.6f\n', D2*1000);
12 fprintf(fid, 'bladeanglesep = %.6f\n', 360/num_blades);
13 fprintf(fid, 'number_blades = %d\n', num_blades);
14 fprintf(fid, 'shaft_dia = %.6f\n', 6); % Adjust as needed
15 fclose(fid);
16
17
18 nx_exe = '"C:\Program Files\Siemens\NX2506\NXBIN\run_journal.exe"';
19 python_journal = '"C:\Users\Emanuel Camacho\Desktop\BLUE\matlabownworkk\FULL\NXpy.py"';
20
21
22 fprintf('\n--- Triggering NX Update Pipeline ---\n');
23 [status, cmdout] = system([nx_exe ' ' python_journal]);
24
25 if status == 0
26     fprintf('SUCCESS: NX part updated and saved.\n');
27     fprintf('Action Required: Open NX and unsuppress the Revolve/Loft features.\n');
28 else
29     fprintf('FAILURE: NX Update crashed.\n');
30     disp(cmdout);
31 end
```

5 Results

The result of this whole process is a much faster design pipeline that when applied to all parts, can lead to a finalized CAD design of an engine after a single MATLAB button click :

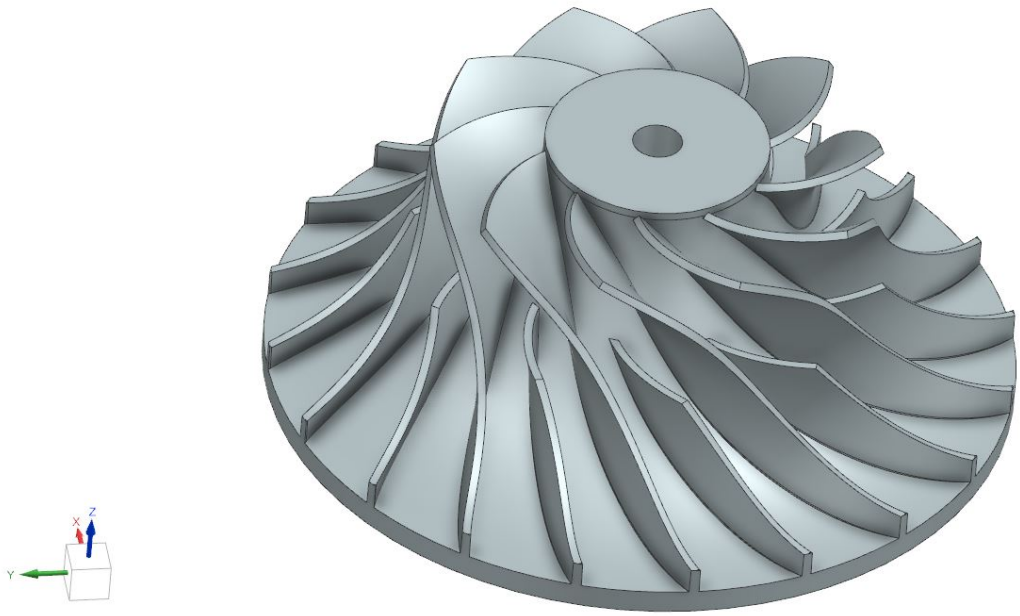

Figure 4: Example of an impeller after reducing the shaft diameter and increasing the number of blades

6 References

References

[1] MathWorks. *system - Execute operating system command and return output*. Accessed: 2026-05-02. n.d. URL: https://www.mathworks.com/help/matlab/ref/system.html.

[2] NX Journaling. *NX Journaling*. Accessed: 2026-05-02. n.d. URL: https://www.nxjournaling.com/.